GPU-Accelerated Parallel BFS: Linear-Algebraic vs. Graph-First on a Single A100

Aakaash Nattanmai | Harshaan Chugh

CS 5220: Applied High-Performance and Parallel Computing • Cornell University • NERSC Perlmutter (NVIDIA A100)

Main finding: On a single A100, the fastest linear-algebraic SpMV BFS kernel outperforms our graph-first direction-optimized BFS on all six tested synthetic graphs. The advantage is smallest on sparse/dense random graphs and largest on chain and power-law graphs, where traversal depth, frontier imbalance, queue overhead, and atomic updates hurt graph-first performance.

1. Motivation

Breadth-First Search is a core primitive in graph analytics, shortest paths, connected components, social-network analysis, and sparse linear algebra workloads. Although BFS is simple on CPUs, it is difficult to optimize on GPUs because the amount of parallel work changes dramatically from level to level.

GPU BFS performance is shaped by:

- irregular memory access,
- changing frontier sizes,
- load imbalance across vertices,
- synchronization and atomic update overhead,
- and graph topology.

This project compares two GPU BFS philosophies on identical hardware and inputs: treating BFS as repeated sparse matrix-vector multiplication versus implementing it as an explicit graph traversal.

2. Two Paradigms

Linear-Algebraic BFS

$$f_{k+1} = (A \cdot f_k) \odot \neg v_k$$

frontier vector × adjacency matrix → next frontier

- Stores the graph as a CSR sparse matrix.
- Each BFS level becomes a sparse matrix-vector operation.
- Different SpMV kernels optimize how edges and frontiers are scanned.

Graph-First BFS

frontier queue → push/pull traversal → next frontier queue

- Stores the graph as adjacency lists plus frontier queues/bitmaps.
- Push scans outward from current frontier vertices.
- Pull scans unvisited vertices checking for frontier neighbors.
- Direction optimization switches between push and pull based on frontier size.

3. Research Questions

1. Which paradigm wins on which graph topology?
2. Are classical optimizations equally beneficial when expressed as SpMV vs. graph traversal?
3. What is the runtime cost of the linear-algebra abstraction compared to hand-tuned graph kernels?

4. Experimental Setup

All experiments run on a single **NVIDIA A100 GPU** on NERSC Perlmutter. Each graph is stored in CSR format and copied to GPU memory before timing. We validate every BFS result against a CPU reference implementation, then report median CUDA kernel time over repeated runs.

Synthetic graphs include:

Synthetic graphs include chain graphs, sparse random graphs, dense random graphs, and power-law graphs.

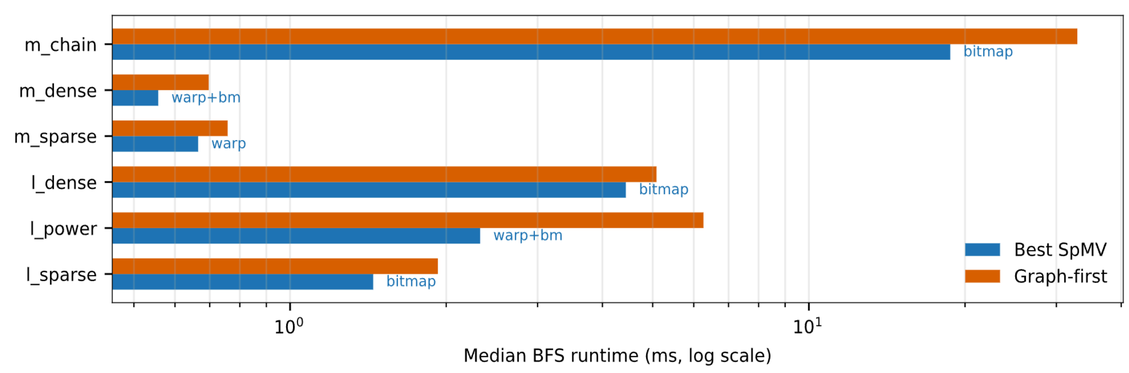
Method Pipeline:

Generate Graph — *Synthetic topology*
↓
Load CSR — *Copy to GPU memory*
↓
Run BFS Kernel — *SpMV or Graph-first*
↓
Validate — *vs. CPU reference*
↓
Record Time — *Median of 10 runs*

Timing excludes graph gen, loading, I/O.

5. Results — Median BFS Kernel Runtime (lower is better)

Median BFS runtime in milliseconds, log scale — Lower is better



Best SpMV Advantage

Graph	Speedup
m_chain	**1.76×**
m_dense	**1.25×**
m_sparse	**1.13×**
l_dense	**1.15×**
l_power	**2.69×**
l_sparse	**1.33×**

Graph-first runtime ÷ Best SpMV runtime

Main result. Across six synthetic graph families on one Perlmutter A100, the fastest SpMV BFS kernel outperforms graph-first direction-optimized BFS in every tested case. The advantage is modest on sparse and dense random graphs, where both approaches expose enough parallelism. The advantage is larger on m_chain and l_power: the chain graph has 999 mostly sequential BFS levels, while the power-law graph creates irregular frontier sizes, load imbalance, and more queue/atomic overhead.

Answers to the Research Questions

Which paradigm wins?
The best SpMV implementation wins or is very close on most tested graphs.

Are all SpMV variants equally useful?
No. The best SpMV kernel depends on graph structure. Bitmap and warp-cooperative variants help in different regimes.

What is the cost of the linear-algebraic abstraction?
In these experiments, the abstraction cost is small compared with topology effects. The hardest cases are not caused by SpMV itself, but by limited parallelism, irregular frontiers, and synchronization/atomic overheads.

6. Linear-Algebraic SpMV BFS

Each BFS level is expressed as a sparse matrix-vector operation over the graph's CSR adjacency matrix. The frontier is represented as either a dense vector or compact bitmap. After each SpMV-style step, already visited vertices are masked out so only newly discovered vertices remain.

We compare four SpMV-style kernels:

Baseline: one thread per vertex.

Warp-cooperative: one warp collaborates on a vertex's neighbor scan.

Bitmap frontier: stores the frontier compactly to reduce memory traffic.

Push-pull: switches between top-down and bottom-up traversal depending on frontier structure.

Same BFS algorithm, different frontier and neighbor-scanning strategies.

7. Graph-First Direction-Optimized BFS

This implementation follows the traditional graph traversal view of BFS. It maintains explicit frontier queues and visited bitmaps, then launches a traversal kernel at each BFS level.

Push mode: scan outgoing edges from vertices in the current frontier.

Pull mode: scan unvisited vertices and check whether any neighbor is already in the frontier.

Direction optimization: switch between push and pull based on frontier size and edge volume.

This approach is intuitive and avoids expressing BFS as matrix-vector multiplication, but it can suffer from queue overhead, atomic updates, irregular memory access, and poor load balance on skewed graphs.

Closer to textbook BFS, but more sensitive to frontier shape and synchronization overhead.

8. Warp-Cooperative Pull

one warp handles one unvisited vertex

```
for each neighbor of the vertex:
  if neighbor is in frontier:
    mark vertex visited
    add to next frontier
    stop scanning
```

Each warp scans one vertex's neighbors in parallel. This reduces per-thread serial work, but performance still depends on frontier density, degree distribution, and atomic updates when vertices enter the next frontier.

✓ Why It Helps
All 32 warp threads stride through the CSR neighbor list in parallel with coalesced reads, and __any_sync lets the warp exit early once any thread finds a frontier hit.

✗ When It Hurts
Low-degree vertices leave most warp lanes idle, and on deep graphs like chains, per-level launch overhead across ~1000 levels outweighs any warp-level gain.

9. Detailed Runtime Tables

Linear-algebraic SpMV (Perlmutter A100; median of 10 runs). Times in ms; bold = best per row.

	$\bar{d}$	dep	Bsln	Wrp	Bmp	P/P	W+B
Medium (10^3 vertices)							
m_chain	2	999	19.55	19.29	**18.73**	32.50	19.22
m_dense	100	2	0.61	2.39	0.60	0.67	**0.56**
m_sparse	6	6	0.69	**0.67**	0.68	0.80	**0.67**
Large (10^5 vertices)							
l_dense	100	4	4.92	5.01	**4.44**	4.81	8.10
l_power	40	3	2.83	8.70	2.47	4.42	**2.33**
l_sparse	10	7	1.71	1.81	**1.45**	1.81	1.58

Graph-first direction-optimizing hybrid. TD/BU counts show how many top-down vs. bottom-up kernel launches the alpha/beta heuristic chose.

	$\bar{d}$	dep	ms	TD/BU
m_chain	2	999	32.92	1000/0
m_dense	100	2	0.70	1/2
m_sparse	6	6	0.76	4/3
l_dense	100	4	5.09	3/2
l_power	40	3	6.26	2/2
l_sparse	10	7	1.93	5/3

10. Future Work

Multi-GPU scaling: extend the comparison to 2 and 4 GPUs per node on Perlmutter.

Better graph-first kernels: improve queue management, bitmap use, and push/pull switching thresholds.

Real-world graphs: evaluate SNAP, GAP, and road-network inputs in addition to synthetic graphs.

Deeper profiling: measure memory bandwidth, atomic contention, warp divergence, and load imbalance.

11. References & Code

[1] S. Beamer, K. Asanović, D. Patterson. *Direction-Optimizing BFS.* SC '12.
[2] D. Merrill, M. Garland, A. Grimshaw. *Scalable GPU Graph Traversal.* PPoPP '12.
[3] J. Kepner et al. *Mathematical Foundations of the GraphBLAS.* HPEC '16.
[4] S. Beamer et al. *The GAP Benchmark Suite.* arXiv:1508.03619, 2015.
[5] NVIDIA RAPIDS. *cuGraph: GPU graph analytics.* github.com/rapidsai/cugraph.

Computations on NERSC Perlmutter (NVIDIA A100, sm_80).
Source, build scripts, validation harness:
`github.com/Harshaan-Chugh/parallel-bfs`